HOTEL RESERVATION SYSTEM

ADR-001 - Feature-First Modular Monolith

Project	Hotel Reservation System
Document	ADR-001 - Feature-First Modular Monolith
Document ID	
Version	
Status	Accepted
Owner	Engineering
Last Updated	2026-09-13

Official readable project artifact. Editable source is maintained under `docs/_source/`.

Document control

Version	Date	Change	Author
1.0	2026-09-13	Initial architecture decision recorded	Engineering

ADR ID

ADR-001

Title

Use a feature-first Modular Monolith for the Express backend

Status

Accepted / LOCKED for V1

Context

The product requires a backend for room inventory, availability, reservations, staff authentication, packages, media, notifications, and future administration. The project starts with one full-time full-stack developer and an urgent delivery goal. The backend needs clear business boundaries and long-term maintainability without the operational and coordination cost of microservices.

A flat layer-first Express structure such as global `controllers/`, `services/`, and `repositories/` would mix unrelated domains as the application grows. At the other extreme, microservices or heavy enterprise patterns would create unnecessary deployment, testing, data-consistency, and operational complexity for V1.

Decision

The Express backend will use a **feature-first Modular Monolith**.

This means:

- One backend application.
- One backend deployment.
- One PostgreSQL database.
- Business-capability modules such as `rooms`, `reservations`, `auth`, `packages`, `media`, and `notifications` as they become necessary.
- Capability-specific code remains inside its owning module.

- Cross-module imports use intentional public capabilities rather than arbitrary access to internal files.
- Shared technical code is introduced only when genuinely cross-cutting.
- The backend does not use a global layer-first business organization.
- Modules do not become independent npm packages or services merely for appearance.

A conceptual module may contain files such as routes, controller, service, repository, schema, and types, but those layers are created **only when the module actually needs them**.

Reasons

1. **Cohesion:** room and reservation behavior can remain understandable within their own capability boundaries.
2. **Low operational cost:** one application and deployment fit the current team size and deadline.
3. **Transactional integrity:** a shared PostgreSQL database supports the strong consistency required to prevent double booking.
4. **Evolvability:** explicit module boundaries reduce the cost of adding auth, packages, media, notifications, and administration later.
5. **Simplicity:** avoids microservice networking, distributed transactions, duplicated infrastructure, and deployment overhead.
6. **Recognizable structure:** experienced Node.js developers can quickly understand the codebase without a custom architecture framework.

Consequences

Positive

- Business code is organized by capability rather than technical layer.
- Cross-domain dependencies become visible during code review.
- The system can share one database and transaction boundary where correctness requires it.
- Deployment and local development remain simple.
- Future extraction of a module is possible if a real scaling/ownership reason appears, without designing for extraction prematurely.

Costs / responsibilities

- Module boundaries are enforced mainly through conventions, code review, and intentional exports unless later tooling is justified.
- Developers must resist moving business logic into generic `shared/` utilities.
- Shared database access requires discipline so one module does not casually mutate another module’s owned data.

- The application must avoid becoming a “big ball of mud” simply because it is a monolith.

Alternatives considered

Flat layer-first Express application

Rejected. Easy initially, but unrelated room, reservation, auth, package, and administration code would become mixed across global folders and harder to navigate as the product grows.

Microservices

Rejected for V1. Adds deployment, network, observability, testing, failure-mode, and data-consistency complexity without a current scale or team-ownership requirement.

Heavy enterprise / clean-architecture framework from day one

Rejected. Generic repositories, DI frameworks, event buses, CQRS, multiple interfaces with one implementation, and similar abstractions are not justified by the present requirements.

Guardrail

The project may revisit this ADR only when a concrete requirement demonstrates that the current architecture no longer meets maintainability, scaling, deployment, compliance, or team-ownership needs. Architectural novelty by itself is not sufficient reason to change it.